IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 2 July 2025, accepted 17 July 2025, date of publication 24 July 2025, date of current version 31 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3592541

RESEARCH ARTICLE

PHILDER: Lightweight Framework for Intelligent Phishing Detection on Resource-Limited Devices

GUILHERME DANTAS BISPO[1], CÉSAR AUGUSTO BORGES DE ANDRADE[1], GABRIELA MAYUMI SAIKI[1], RAQUEL VALADARES BORGES[2], ANDRÉ LUIZ MARQUES SERRANO[1], GERALDO PEREIRA ROCHA FILHO[3], AND VINÍCIUS PEREIRA GONÇALVES[1]

[1]Department of Electrical Engineering, University of Brasília (UnB), Brasília 70910-900, Brazil
[2]Department of Statistics, UnB, Brasília 70910-900, Brazil
[3]Department of Exact and Technological Sciences, State University of Southwest Bahia (UESB), Vitória da Conquista 45083-900, Brazil

Corresponding author: Guilherme Dantas Bispo (guilherme.bispo@redes.unb.br)

ABSTRACT The increasing sophistication of phishing attacks represents a significant challenge to cybersecurity, particularly in the context of fraudulent emails crafted to deceive users and steal confidential information. This paper proposes a deep learning-based approach for the automatic detection of phishing in emails, the PHILDER (PHishing Intelligent Lightweight DEtection on Resource-limited devices) model. The study employs a dataset composed of real emails extracted from PhishTank and the SpamAssassin Public Corpus, spanning both legitimate and fraudulent samples. To address the natural imbalance between spam and phishing emails, two strategies were applied: undersampling, which reduces the number of legitimate emails to balance the classes, and oversampling using SMOTE (Synthetic Minority Oversampling Technique) to generate new synthetic samples of the minority class. Five models (ALBERT, DistilBERT, MobileBERT, MiniLM, and TinyBERT) were trained: one using the original dataset, one with undersampling, and a third with oversampling. The experimental results show that the oversampling approach had to be discarded due to computational cost issues, the tendency to overfit, and inferior performance compared to the other two approaches. The main contributions of this paper include the application of computational efficiency metrics for model evaluation, which not only consider traditional metrics but also enable assessment in real-world applications under hardware computational constraints, and the exploration of different data composition strategies for training. The use of PHILDER (TinyBERT combined with data balancing techniques) demonstrates a promising approach for the automatic detection of phishing in emails, contributing to strengthening digital security against social engineering attacks.

INDEX TERMS Phishing, fraud detection, deep learning, BERT, cybersecurity.

I. INTRODUCTION

The advancement of networking and communication technologies has profoundly transformed modern society, providing greater connectivity and facilitating essential activities such as social media interactions, banking transactions, and e-commerce [1]. However, the same infrastructure that enables this connectivity also introduces vulnerabilities exploited by cybercriminals. Among cyberattacks, *phishing* stands out for its ability to deceive users and compromise sensitive information, becoming one of the most prevalent and sophisticated attacks today [2]. This growing threat has driven research aimed at developing more effective detection and mitigation methods.

Phishing is a social engineering-based attack in which criminals create fake pages that mimic legitimate sites to obtain access credentials, financial data, and personal information. Although experienced users can identify malicious URLs, factors such as haste and lack of attention contribute to the success of these attacks [1]. Furthermore, studies indicate that attackers exploit users' psychological and behavioral characteristics, increasing the effectiveness

The associate editor coordinating the review of this manuscript and approving it for publication was Laura Celentano.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

of fraud attempts [3]. These tactics include creating a sense of urgency, threats of account suspension, and persuasive messages that induce victims to err. The financial impacts of these attacks are significant, affecting both individuals and large organizations. According to McAfee (2020), annual global losses due to cybercrime can reach US$1 trillion, with *phishing* being a central threat to both companies and consumers.

In light of this scenario, various approaches for *phishing* detection have been explored, ranging from rule-based methods and blocklists to more advanced machine learning and deep learning techniques. Traditional methods, which analyze known attack patterns, exhibit limitations in detecting new variations, requiring constant updates and refinement. In contrast, deep learning-based techniques have shown great potential in analyzing subtle characteristics of malicious emails and URLs, allowing for more effective detection even against innovative attacks [4]. Additionally, methods based on email or URL metadata are widely used but also require continuous improvements to keep up with evolving *phishing* tactics.

Textual content analysis of emails has emerged as a promising approach for *phishing* detection, since many fraudulent messages share linguistic and structural patterns that can be automatically identified. Neural network models such as Recurrent Neural Networks (RNNs) and Transformers have been studied to improve the accuracy of malicious email classification, enabling more efficient differentiation between legitimate and fraudulent messages. Hybrid approaches that combine text analysis, metadata, and URL features have also been explored to build more robust and adaptable detection systems. With the rapid evolution of cyber threats, the continuous development of new strategies for *phishing* detection becomes essential. Attackers' adaptation to traditional countermeasures demands that security systems also evolve, incorporating more advanced and flexible techniques. Thus, research in the field of fraudulent email detection remains fundamental to ensure higher security in digital communication and minimize risks associated with *phishing* attacks.

Email *phishing* is a cyberattack that can have severe consequences, including financial loss, identity theft, and reputational damage to an organization. The number of *phishing* victims has increased drastically over the last decade, with millions affected each month. These attacks are difficult to detect and have catastrophic potential in the business finance domain, potentially causing millions in losses to organizations. According to [5], approximately 15% of breaches occur via *phishing*, with an average cost of US$4.88 million per incident, which can take up to 261 days to detect.

Organizations must be able to identify and block these attacks to avoid falling victim to this threat. [6] defines a *phishing* attack as fraud that occurs when a fake site mimics a real site to acquire unsuspecting users' data. A duplicated site resembling a legitimate one is created for *phishing*, making detection difficult [7]. Currently, *phishing* is a widely known issue, and the impact of *phishing* attacks has become disastrous [8]. *Phishing* attacks have put Internet users at risk [6], who increasingly have their personal data easily accessed and become more susceptible to virtual scams.

A clear upward trend is observed in the number of phishing attacks throughout the third quarter of 2024. In July, reports were around 290,000 occurrences, rising to approximately 300,000 in August. September saw a significant increase, with over 340,000 attacks reported. These data, extracted from the trend report published by the Anti-Phishing Working Group (APWG) [9], highlight an intensification of *phishing* activities, indicating a worsening of cyber threats during the period. The chart clearly illustrates this evolution, emphasizing consistent and steep growth over the three months analyzed.

Therefore, email *phishing* continues to be one of the main threats to digital security, affecting individuals and organizations globally. The sophistication and diversity of attackers' techniques demand equally advanced and adaptable defense strategies. In this context, deep neural network models have the potential to deliver promising results, especially due to their ability to learn complex contextual representations directly from data without extensive manual intervention [10]. However, the practical deployment of these models often faces challenges related to scalability and computational efficiency, particularly in environments requiring quick responses or operating under hardware constraints.

However, the practical deployment of these models often faces significant challenges, particularly in environments with limited computational resources, such as IoT devices, mobile phones, or embedded systems. Traditional deep learning models, especially those based on large Transformer architectures, typically demand substantial processing power (CPUs, GPUs, TPUs) and memory, making their direct implementation on resource-constrained devices impractical or inefficient. This limitation arises from several factors:

- High Computational Cost: Complex neural networks involve millions or even billions of parameters, requiring extensive matrix multiplications and convolutions during inference. This translates to high CPU/GPU cycles and energy consumption, which are scarce resources in edge devices.
- Large Memory Footprint: Storing model parameters and intermediate activations consumes significant memory. Resource-constrained devices often have limited RAM and storage, making it impossible to load and run large models.
- Latency Requirements: Many real-time cybersecurity applications, such as phishing detection, require low-latency responses. Large models can introduce considerable inference delays, rendering them unsuitable for scenarios where immediate action is critical.

- Energy Consumption: Continuous operation of computationally intensive models drains battery life quickly in portable or IoT devices, which is a major concern for practical deployment.
- Model Complexity and Maintenance: While powerful, these models can be closed boxes, making debugging, interpretability, and updates challenging, especially in dynamic threat landscapes.

These limitations highlight a critical gap in current phishing detection solutions: the trade-off between detection accuracy and operational efficiency on resource-constrained devices. While highly accurate models exist, their computational demands often preclude their use in environments where they are most needed, leaving a significant portion of potential victims vulnerable. This necessitates the development of lightweight, yet effective, models specifically designed for such environments.

Studies in the field demonstrate that highly effective *phishing* detection can be achieved using only the textual body of emails as the data source. Approaches combining Natural Language Processing (NLP) techniques such as lemmatization and Bag-of-Words (BoW) vectorization with topic models like Latent Dirichlet Allocation (LDA) have shown excellent performance in identifying discriminative patterns, even with highly sparse datasets. These strategies circumvent issues like the ''curse of dimensionality'' and lack of semantic context by offering compact, expressive feature vectors. Gualberto et al. [11], for instance, achieved an F1-score of 99.95% with the XGBoost algorithm using exclusively the email body, without relying on metadata or links, demonstrating that well-designed feature engineering solutions can rival more complex models.

Despite the success of these approaches, many still require multiple preprocessing steps, feature selection, or topic extraction, which can increase implementation complexity in real-world environments. In this context, the present work proposes a BERT-based model (Bidirectional Encoder Representations from Transformers), which eliminates the need for manual feature engineering and provides robust contextual representations directly from email text. Moreover, this study investigates the impact of class balancing techniques such as undersampling and SMOTE on model performance, a topic often overlooked in previous Transformer-based research. By combining a pretrained language model with data imbalance handling strategies, this approach aims to offer a practical, scalable, and accurate solution for automatic phishing detection in emails, making a significant advancement over previous NLP-based or manually engineered supervised models.

In this work, we propose the PHILDER (PHishing Intelligent Lightweight DEtection on Resource-limited devices) model, a BERT-variant-based approach aimed at automatic detection of phishing emails in resource-constrained scenarios. Using exclusively message textual content and exploring data balancing techniques such as undersampling and SMOTE, the research evaluates five BERT-based model variations in terms of predictive performance and computational efficiency. Through the composite metrics TES, IES, and RTDE, we demonstrate that the PHILDER model, derived from TinyBERT with undersampling, offers the best balance between accuracy and computational cost, standing out as a practical, scalable, and highly effective solution for real-world cybersecurity applications.

The paper is structured as follows: Section II presents a detailed review of related work on phishing detection using deep learning and Transformer-based techniques. Section III describes the PHILDER methodology, as well as materials and methods employed, detailing datasets, preprocessing techniques, and balancing strategies. Section IV discusses the experimental results. Finally, Section V provides concluding remarks, highlighting key findings and contributions.

II. RELATED WORK

The identification of related work followed the approach described by Bispo [12]. A search was conducted in the *Scopus* database using the keywords *phishing detection*, restricted to the Computer Science area, and considering only publications from the last five years. This resulted in 839 articles, of which the ten most relevant were selected for analysis.

This section organizes the related work based on the characteristics of the phishing detection techniques employed, primarily categorizing them into Machine Learning and Deep Learning approaches. This classification aims to provide a clearer understanding of the methodologies and the current landscape of research in the field.

A. MACHINE LEARNING APPROACHES

Various studies have explored the use of machine learning algorithms for effective phishing detection, leveraging these models' ability to identify complex patterns in large volumes of data. The application of Natural Language Processing (NLP) techniques in conjunction with these algorithms has proven particularly promising for analyzing textual and behavioral characteristics of URLs and emails.

The most cited paper proposes a *phishing* detection system based on seven machine learning algorithms: *Decision Tree*, *Adaboost*, *K-star*, kNN, *Random Forest*, SMO, and *Naive Bayes* [1]. The system analyzes a set of features that includes NLP techniques, which showed superior performance compared to simple word vectors. The combination of these approaches resulted in a significant increase in model accuracy. The results reinforce the effectiveness of applying machine learning algorithms in conjunction with NLP techniques for *phishing* detection, highlighting the importance of synergy between advanced analysis methods and predictive models to improve accuracy and efficiency in identifying fraudulent sites.

The work by [13] proposes the *Hybrid Ensemble Feature Selection* (HEFS), a novel approach for feature selection in *phishing* detection systems based on machine learning.

HEFS comprises two phases: first, the *Cumulative Distribution Function gradient* (CDF-g) algorithm generates feature subsets, which are refined by a data perturbation *ensemble*. In the second phase, features undergo further refinement via a function perturbation *ensemble*. Tests demonstrate that HEFS, combined with a *Random Forest* classifier, achieves high accuracy using a reduced number of features, outperforming classifiers such as SVM and *Naive Bayes*. The methodology also proved effective on an additional UCI dataset. HEFS offers an efficient, flexible, and automated solution for enhancing *phishing* detection, with potential application across diverse datasets. Future research is recommended to remove obsolete features and explore new classification algorithms.

The study by [14] presents four advanced models for fraudulent site detection based on machine learning: *AdaBoost-Extra Tree*, *Bagging-Extra Tree* (BET), *Rotation Forest-Extra Tree* (RoFBET), and *LogitBoost-Extra Tree* (LBET), all using the robust *Extra-tree* classifier. The main goal was to improve *phishing* identification by reducing false positives and negatives. The models exhibited excellent performance, with high detection accuracy and significantly low false alarm rates. Compared to traditional approaches, they proved more effective, overcoming limitations of hybrid methods and conventional classifiers. The central innovation lies in the meta-learning application, offering greater result interpretability than more complex solutions. The study also emphasizes model transparency, facilitating future adjustments and strengthening trust in real-world implementations. Future work should explore other decision tree techniques for further optimization.

The research by [15] investigates AI applications in *phishing* site detection using URL features such as domain identity, anomalies, and HTML/JavaScript elements. The aim was to assess how these features can enhance site classification effectiveness. Two datasets with over 48 semantic features were used, and AI models were evaluated by precision, *recall*, and AUC metrics. Sixteen algorithms were tested, with *GradientBoostingClassifier* and *RandomForestClassifier* performing best, while *GaussianNB* and SGD underperformed. The study suggests that combining learning techniques like *bagging* and *boosting* can improve *phishing* detection accuracy. Future research should include more variables and hybrid approaches to optimize predictive capabilities and reduce errors.

The study by [16] used a phishing site dataset from the UCI Machine Learning Repository, comprising 30 technical and behavioral site features. Accuracy was the main performance metric. The proposed model outperformed existing technologies, demonstrating high accuracy. Feature selection was applied to reduce the dataset to the most relevant variables, ensuring irrelevant attributes did not compromise prediction quality. An ensemble learning technique combined different learning models, with final decisions made by majority voting to avoid single-classifier bias.

Between 2006 and 2016, email classification studies focused on spam and phishing identification and message categorization. Datasets such as PU, PhishingCorpus, and Enron were widely used, exploring header, content, and link features. Supervised learning methods, particularly SVM, emerged as the most promising approach, with metrics like accuracy, recall, precision, and F1-score employed for performance evaluation. Despite progress, challenges remain, including new attack types and the need for better datasets. Future research may extend analysis to emails in other languages and classify different message types [17].

B. DEEP LEARNING APPROACHES

Deep learning, with its complex neural network architectures, has revolutionized phishing detection, enabling multi-layered data analysis and the extraction of high-level features that would be difficult to identify with traditional methods. These approaches are particularly effective in detecting sophisticated and constantly evolving threats.

In the next paper, the authors propose a cloud-based solution to strengthen Internet of Things (IoT) device security by detecting both *phishing* and *Botnet* attacks [18]. The method employs two complementary security models: a Deep Convolutional Neural Network (DCNN), implemented directly on IoT devices to identify *phishing* and DDoS attacks, and an LSTM model in the *backend*, responsible for detecting Botnets and using data extracted by the DCNN to recognize distributed *phishing* attacks. Models were trained with datasets containing *phishing* and non-phishing URLs, as well as botnet attack specifics. Results show that the DCNN efficiently detected *phishing* on IoT devices, while the LSTM performed well in backend Botnet identification. Model integration optimized resource usage, minimized communication needs, and enabled more effective distributed attack detection, reinforcing the robustness of the proposed approach.

The study [19] proposes a model combining Graph Convolutional Networks (GCN) with NLP to detect phishing emails by analyzing email body content, an area often overlooked by traditional detection methods. The model achieved high true positive rates and low false positives, excelling in metrics such as true positive rate, true negative rate, recall, and precision, outperforming other deep learning approaches. It highlights the importance of analyzing email body text to improve accuracy and shows promise for detecting new and emerging phishing threats.

Tang and Mahmoud [20] present a real-time phishing site detection solution implemented as a user-friendly browser extension. The proposal stands out for its independence from external services and high-quality data collected via user feedback, enabling continuous model improvement. Key features include instant alerts for suspicious pages, a combination of site blocklist checks and machine learning to reduce false alarms, and logging for ongoing enhancements.

TABLE 1. Consolidation of related work.

Paper	Problem	Technique	Data	Metrics	Accuracy
[1]	URL	NLP-based features	Openfish, PhishTank	Accuracy, Precision, Recall (Sensitivity), F1-Score, ROC-AUC, Confusion Matrix	97.98%
[18]	URL	Feed Forward Neural Networks (FFNN)	PhishTank (OpenDNS, 2016) and OpenPhish	Accuracy, Sensitivity, Specificity, AUC-ROC	94.80%
[13]	URL	Hybrid Ensemble Feature Selection (HEFS)	UCI Phishing Dataset	Accuracy, Sensitivity, F1-Score, AUC-ROC, True Positive Rate, False Positive Rate	94.60%
[14]	URL	Deep Learning	PhishTank, Phishing Websites	Accuracy, False Positive Rate, False Negative Rate, AUC-ROC	97%
[15]	URL	Deep Learning	Phishing Websites	Accuracy, Recall, F1-Score, AUC-ROC	97%
[16]	URL	Ensemble Learning	Phishing Website Detection	Accuracy, Precision, Recall, F1-Score, ROC-AUC	95%
[19]	Email	NLP	Ham dataset	Accuracy, Precision, Recall, F1-Score	98.20%
[20]	URL	Deep Learning (RNN-GRU)	Phishing Website Detection	Accuracy, Recall, F1-Score, AUC-ROC	99.18%
[17]	Email	Deep Learning	Spambase, Enron, Phishing Websites, SMS Spam Collection	Accuracy, Recall, F1-Score, AUC-ROC, False Positive Rate	–
[21]	Email	Deep Learning (RNN)	Catching the Phish	Accuracy, Recall, F1-Score	–
This work	Email	Deep Learning	PhishTank, SpamAssassin Public Corpus	Accuracy, Precision, Recall, F1-Score, TES, IES, RTDE	99.27%

Future work includes cloud deployment with GPU support, expansion to other browsers, and integration of technologies like computer vision to further improve efficiency and reliability.

The paper by [21] proposes an RNN-based phishing email detection solution focusing on email text structure. Instead of relying on predefined features, the system learns phishing patterns automatically, making it more flexible and adaptable. Results indicated that this approach outperforms traditional methods and can constantly update to capture new phishing forms. The technique can be extended to websites, offering a more robust and automated way to combat phishing and spam attacks.

The information from related work has been consolidated in Table 1. The table summarizes the main studies analyzed, highlighting the addressed problem, employed technique, datasets used, and evaluation metrics adopted. This consolidation provides an overview of recent literature approaches, facilitating comparison between different methods and their respective performance.

After consolidating related work in Table 1, we observe that most research focuses on URL-based *phishing* detection, exploring various machine learning and deep learning techniques. However, a gap exists in email-based deep learning approaches evaluated by computational efficiency metrics (TES, IES, and RTDE). While some studies address this problem, the majority use traditional NLP techniques or less sophisticated models and rely primarily on traditional evaluation metrics. Therefore, there is an opportunity to delve deeper into advanced deep learning models for email analysis, exploring neural architectures that can enhance accuracy, efficiency, and robustness in *phishing* detection for this specific category.

III. PHILDER: PHISHING INTELLIGENT LIGHTWEIGHT DETECTION ON RESOURCES-LIMITED DEVICES

This section describes the methodology for automatic phishing detection in emails. Figure 1 presents the methods and procedures used to carry out phishing detection research in emails through natural language processing models. The advancement of natural language models has enabled the application of sophisticated threat detection techniques in electronic communications such as email phishing. However, using large models can be impractical in hardware-constrained or real-time environments. In this context, lightweight BERT-derived models have emerged as promising alternatives.

A. DATABASE CONSOLIDATION

For the development of an email phishing detection model based on supervised artificial intelligence, it is essential to have a pre-labeled dataset composed of representative examples of both legitimate and fraudulent emails. In this work, two publicly available datasets widely recognized in the scientific literature were employed:

- **PhishTank:** A collaborative platform maintained by the OpenDNS community, dedicated to the collection and verification of URLs and emails used in phishing attacks. Samples are submitted by users and subsequently validated through community voting. In this study, 501 confirmed malicious emails were utilized, exhibiting typical characteristics of phishing campaigns—such as attempts to steal banking credentials, service identity spoofing, and redirection to fraudulent web pages. The fields available in this dataset include: sender, subject, email body (text), suspicious URLs, and the class label (*phishing*).

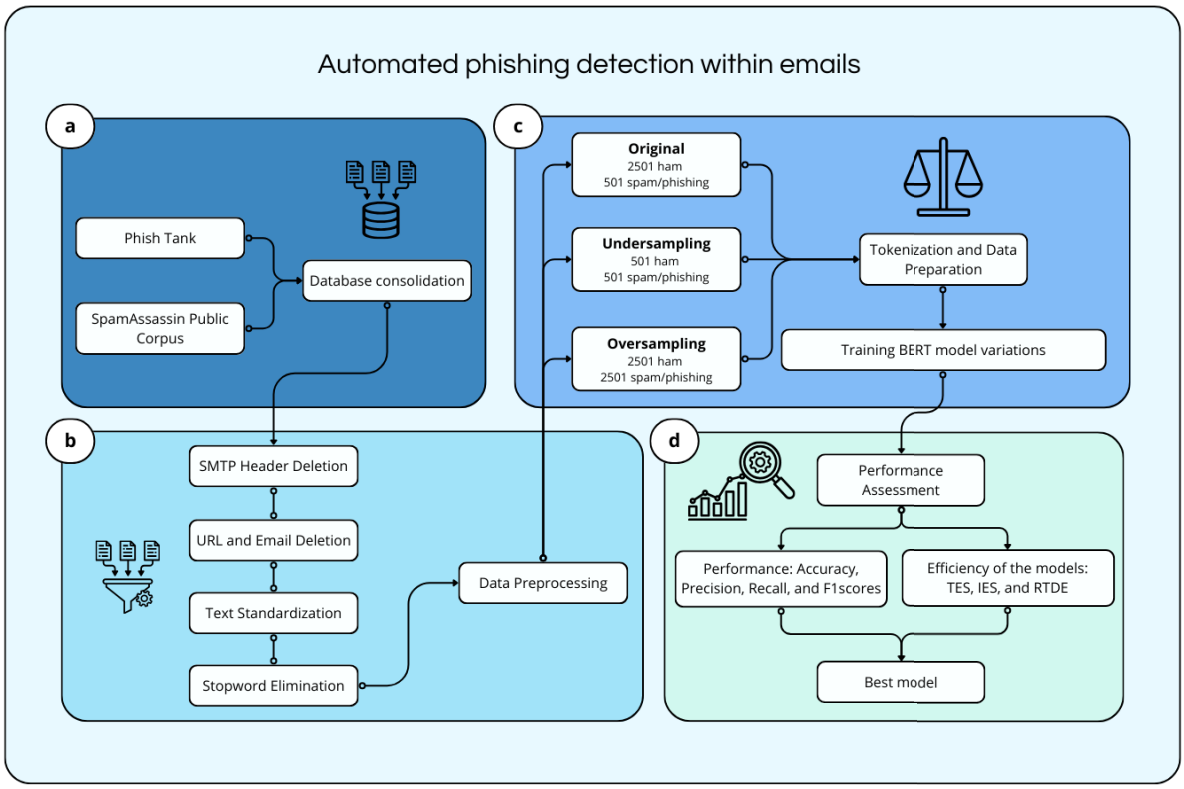


FIGURE 1. Phishing detection research methodology.

- **SpamAssassin Public Corpus:** A dataset maintained by the Apache SpamAssassin project, extensively used for research in spam filtering and email classification. The corpus includes *ham* (legitimate) messages collected from technical mailing lists and user groups, preserving full content—including metadata and the email body. In this study, 2,501 legitimate emails were employed. The primary fields considered for analysis include: sender, subject, email body (text), SMTP headers, and the class label (*ham*).

The consolidation of these two sources resulted in an initial dataset comprising 3,002 emails, of which 501 were classified as *phishing* and 2,501 as legitimate. The selection of these datasets was motivated by their broad acceptance within the scientific community, public availability, and the diversity of samples representative of real-world attacks and authentic communications.

This combination enabled the creation of a robust dataset, essential for the training and validation of supervised models, thereby ensuring strong generalization capabilities in practical contexts. Furthermore, the clear distinction between malicious and legitimate emails renders the dataset well-suited for binary classification tasks using *transformer*-based techniques.

B. DATA PREPROCESSING

The preprocessing of emails is a crucial step to ensure that the data are structured efficiently for the Deep Learning model. In this phase, we applied a set of techniques to remove noise and standardize the text.

- **SMTP Header Removal:** Only the email body was retained, eliminating sender, recipient, intermediate server information, and routing metadata.
- **URL and Email Address Removal:** Email addresses and links were replaced with generic placeholders to prevent the model from learning specific patterns that do not generalize well.
- **Text Normalization:** Content was converted to lowercase and special characters were removed, ensuring uniformity in the model input.
- **Stopword Removal:** Words irrelevant to classification, such as pronouns and conjunctions, were removed to reduce data dimensionality.

After this stage, a *DataFrame* was constructed containing the cleaned texts and their respective classes (0 for *ham* and 1 for *spam/phishing*), ready to be processed by the balancing strategies.

C. DATABASE BALANCING

The original dataset exhibited a significant imbalance, containing 2,501 legitimate (*ham*) emails and 501 emails classified as *spam/phishing*. Deep learning models tend to be influenced by the majority class, impairing the detection of fraudulent emails. To mitigate this issue, we adopted two distinct balancing strategies.

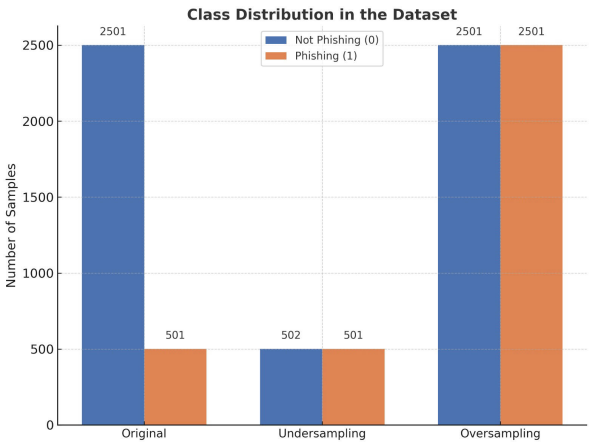


FIGURE 2. Data distribution in the dataset.

In the undersampling technique, we reduced the number of majority-class (*ham*) emails to balance them with those of the minority class (*spam*/*phishing*). To this end, we randomly selected 502 legitimate emails while retaining the 501 *spam*/*phishing* emails. This approach seeks to prevent majority-class bias but may lead to the loss of valuable information due to the reduction of legitimate data.

For the oversampling strategy, we employed the Synthetic Minority Over-sampling Technique (SMOTE), which synthesizes new minority-class samples based on the existing distribution. However, since SMOTE operates solely on numerical data, it was necessary first to convert the email texts into vectors using TF-IDF (*Term Frequency-Inverse Document Frequency*). After applying SMOTE, we obtained a final dataset containing 2,501 legitimate emails and 2,501 *spam*/*phishing* emails. The class distribution in the dataset is shown in Figure 2.

After balancing, email texts were converted into numerical representations compatible with the BERT-base-uncased model. We used the standard BERT tokenizer, which segments the text into tokens and applies padding to standardize sequence lengths.

The data were then split into 70% training and 30% testing sets using the `datasets` library from Hugging Face to ensure compatibility with the model. For each balancing approach, we created independent training and testing sets.

A large proportion of lightweight models employed in Natural Language Processing (NLP) tasks—such as phishing email detection—are derived from the BERT architecture (Bidirectional Encoder Representations from Transformers).

BERT is a pre-trained NLP model designed to understand the context surrounding each token. Released by Google AI in 2018, it is widely accessible for various applications. Unlike earlier NLP models that only considered left-context, BERT's bidirectional and unsupervised pre-training allows it to learn from both left and right contexts using simple text corpora [22]. Its application of the Masked Language Model (MLM) represented a significant advance in NLP, offering a straightforward implementation compared to other models and proving to be a powerful tool for language understanding [23].

Given the limitations of unidirectional models, which only consider preceding context, BERT's bidirectional design offers superior interpretation of text, context, and even sentiment—hence its designation as a masked model capable of predicting bidirectional context. BERT operates in two main phases: pre-training and fine-tuning [22].

During pre-training, BERT performs two tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). In the MLM task, the model predicts the original tokens that were masked, leveraging the context of the surrounding words. This deep learning process requires the model to infer the meaning and context of words to accurately predict the masked tokens. In the NSP task, the model learns to predict sentence order by being given pairs of sentences and determining whether the second sentence follows the first, thereby enhancing its understanding of inter-sentence relationships. After pre-training, BERT is fine-tuned on task-specific datasets, which refines and improves its performance. Architecturally, BERT is built upon the Transformer framework, comprising multiple encoding layers [22].

As the foundation for many derivative models, BERT demonstrates strong performance in text inference and classification by effectively capturing sentence relationships and contextual nuances. In cybersecurity contexts—such as analyzing and classifying email content, URLs, and web pages to detect patterns of fraudulent attacks—BERT and its variants have become essential tools.

To identify the best-performing variant, we tested five BERT-based models that retain the core Transformer and bidirectional attention mechanisms but incorporate structural modifications or compression techniques to reduce computational complexity. DistilBERT uses knowledge distillation to achieve approximately 97% of BERT-base's performance with only 60% of its parameters [24]. TinyBERT follows a similar two-stage distillation approach with even fewer layers, making it ideal for hardware-constrained environments [25]. MobileBERT introduces bottleneck layers and linear projections to optimize execution on mobile devices [26]. ALBERT employs parameter sharing and embedding factorization to drastically reduce parameter count without significant performance loss [27]. Finally, MiniLM adopts a more compact self-attention mechanism, delivering high accuracy with notable computational efficiency [28]. These models strike an excellent balance between performance and computational cost, making them particularly suitable for real-time and scalable security applications in resource-limited settings.

Each model exhibits distinct architectural features, making some variants more advantageous depending on the deployment scenario. Table 2 provides a comparative overview of the main differences among the tested BERT models.

The TinyBERT model is the most compact among the BERT variants evaluated in this study. This is because it

TABLE 2. Summary of BERT model variations.

Model	Objective	Modifications
ALBERT [27]	Reduce memory consumption and improve model speed	Parameter reduction and replacement of the BERT NSP loss with Sentence-Order Prediction (SOP).
MobileBERT [26]	Optimize BERT comprehension and acceleration for mobile devices	Bottleneck architecture, inverted-bottleneck knowledge transfer, and operational optimizations to lower latency.
DistilBERT [24]	Produce a smaller, faster BERT requiring fewer resources	Enhanced pre-training via knowledge distillation, including student architecture, student initialization, and triple loss.
TinyBERT [25]	Develop a compact BERT through a two-stage learning process	Transformer distillation with attention-based (L_{attn}), hidden-states-based (L_{hidn}), and prediction-layer (L_{pred}) losses.
MiniLM [28]	Compact the Transformer architecture	Deep self-attention distillation leveraging knowledge from the final layer.

was designed specifically to be a leaner and more efficient version of the BERT model. Analogous to DistilBERT, TinyBERT relies on the Knowledge Distillation (KD) technique applied to Transformer models, which seeks to transfer the knowledge acquired by the full-sized BERT into a smaller network, TinyBERT. The distillation process employs three loss functions: Attention-Based Distillation (L_{attn}), Hidden States-Based Distillation (L_{hidn}), and Prediction-Layer Distillation (L_{pred}) [25].

TinyBERT's training proceeds in two stages. In the first, general distillation stage, TinyBERT learns the intermediate representations of the original BERT during pre-training. At this stage, Prediction-Layer Distillation (L_{pred}) is omitted, as it yields no significant benefit. In the second, task-specific distillation stage, the fine-tuned BERT model serves as the teacher for TinyBERT, guiding the distillation of the Transformer layers. Additionally, training data are augmented using GloVe word embeddings to identify semantically related words [25]. The overall knowledge transfer is formalized by the following loss function:

$$L_{\text{model}} = \sum_{x \in X} \sum_{m=0}^{M+1} \lambda_m L_{\text{layer}}\left(f_m^S(x),\, f_{g(m)}^T(x)\right) \tag{1}$$

Here, L_{model} denotes the total loss, x is a text input, X the training corpus, m the index of a layer in the smaller model, M the number of Transformer layers in TinyBERT, N the number of Transformer layers in the larger model, $g(m)$ the mapping function from student to teacher layers, λ_m the hyperparameter weighting the importance of each layer's distillation, and $L_{\text{layer}}(f_m^S(x),\, f_{g(m)}^T(x))$ the layer-specific loss between student and teacher representations.

Architecturally, TinyBERT can be instantiated with varying numbers of Transformer layers and hidden-dimension sizes, depending on application requirements. Despite its extreme compactness and speed, it achieves competitive performance relative to other natural language processing approaches [25].

D. PERFORMANCE ASSESSMENT

The evaluation of the trained models was conducted in two stages. The first stage was based on four primary metrics: accuracy, precision, recall, and F1-score. The purpose of this evaluation is to determine which model delivers the best performance in phishing-email detection. The second stage addresses each model's computational efficiency, since some models may achieve excellent phishing detection performance at the expense of high computational cost; in practical deployments, it is therefore preferable to select models that offer comparable detection performance while remaining computationally efficient.

The metrics are defined as follows: Accuracy, which measures the overall proportion of correct classifications made by the model; Precision, which represents the rate of true positives among all phishing predictions made by the model; Recall, which measures the model's ability to correctly identify all phishing emails; and F1-score, which computes the harmonic mean of precision and recall, providing a more robust measure in scenarios with class imbalance.

The computational efficiency metrics were assessed in three stages. The first is the Training Efficiency Score (TES): this metric denotes the ratio of model accuracy to training time (Accuracy / Training Time). TES indicates how effectively a model can learn under a constrained training budget—a requirement essential for systems that must be retrained frequently or deployed on resource-limited devices.

The second is the Inference Efficiency Score (IES): defined as the ratio of accuracy to total testing time (Accuracy / Testing Time), IES measures the model's efficiency when applied to unseen data. This metric is especially relevant for evaluating batch-processing detection pipelines, common in real-world email-filtering systems.

Finally, the third metric is Real-Time Detection Efficiency (RTDE): calculated as (Accuracy / Detection Time per Sample), RTDE reflects the model's capability to deliver accurate predictions under strict latency constraints—a critical factor in real-time phishing detection within secure email gateways or client-side anti-phishing modules.

The results obtained for the DistilBERT, ALBERT, MiniLM, MobileBERT and TinyBERT models according to these metrics are presented in Section IV.

TABLE 3. Model evaluation results.

Model	Acc.	Prec.	Rec.	F1	Train Time (s)	Test Time (s)	Detect Time/ sample (ms)
DistilBERT (1)	0.9922	0.9815	0.9755	0.9785	328.48	13.33	14.7906
DistilBERT (2)	0.9934	0.9866	1.0000	0.9932	111.83	4.48	14.8841
DistilBERT (3)	0.9960	0.9947	0.9973	0.9960	546.68	22.19	14.7839
ALBERT (1)	0.9845	0.9668	0.9448	0.9557	702.01	29.60	32.4928
ALBERT (2)	0.9834	0.9797	0.9864	0.9831	236.81	9.09	32.8417
ALBERT (3)	0.9913	0.9934	0.9874	0.9904	368.87	12.38	33.2564
MiniLM (1)	0.9936	0.9739	0.9688	0.9713	305.42	16.63	18.4824
MiniLM (2)	0.9889	0.9857	0.9982	0.9919	103.52	6.21	16.1503
MiniLM (3)	0.9931	0.9922	0.9989	0.9956	504.74	23.40	17.0932
MobileBERT (1)	0.9843	0.9593	0.9609	0.9601	648.99	26.08	22.5134
MobileBERT (2)	0.9904	0.9781	0.9865	0.9823	219.45	9.76	22.5147
MobileBERT (3)	0.9916	0.9871	0.9900	0.9885	860.67	43.41	22.5171
TinyBERT (1)	0.9936	0.9938	0.9930	0.9934	212.39	7.61	12.9966
TinyBERT (2)	0.9927	0.9896	0.9960	0.9927	180.52	6.22	12.9811
TinyBERT (3)	0.9973	0.9973	0.9973	0.9973	337.91	12.71	12.9860

IV. RESULTS OF MODEL PERFORMANCE

Table 3 presents the values obtained for the metrics accuracy, precision, recall, F1-Score, training time (s), testing time (s), and detection time per sample (ms) for the four trained models, considering the original (1), undersampling (2), and oversampling (3) approaches.

Considering accuracy, precision, and F1-score, the TinyBERT oversampling model achieved the best performance. For recall, the DistilBERT undersampling model performed best; although its recall equals 1.0000, which may indicate overfitting, its strong accuracy, precision, and F1-score values suggest genuine effectiveness in this regard. The original MobileBERT model exhibited the poorest results in precision, recall, and F1-score, while the MobileBERT undersampling variant had the lowest accuracy.

The ALBERT oversampling model required the longest training time (958.35 s), followed by MobileBERT oversampling (880.67 s), original ALBERT (702.01 s), original MobileBERT (648.99 s), DistilBERT oversampling (546.68 s), and MiniLM oversampling (504.74 s). For testing time, the TinyBERT undersampling model was the fastest (2.57 s), whereas ALBERT oversampling was the slowest (43.58 s), followed by MobileBERT oversampling (43.41 s). In terms of detection time per sample, the original ALBERT model required the most time (32.8483 s), while the TinyBERT undersampling variant required only 12.9811 s.

To further contextualize PHILDER's performance, we conducted a comparative analysis with recent studies on phishing detection. Although direct comparisons are challenging due to variations in datasets, evaluation metrics, and experimental setups, our goal is to demonstrate PHILDER's competitive advantage, especially regarding its lightweight nature and efficiency on resource-constrained devices.

It is observed in Table 1 that many studies focus on URL-based phishing detection. For example, Sahingoz et al. [1] achieved 97.98% accuracy with NLP-based features for URLs. Tang and Mahmoud [20] reported 99.18% accuracy using Deep Learning (RNN-GRU) for phishing detection in URLs. Although these results are high, they focus on URLs, while PHILDER excels in email phishing detection.

For email phishing detection, Alhogail and Alsabih [19] obtained 98.20% accuracy using NLP. PHILDER, with 99.27% accuracy, surpasses this result, demonstrating superior performance in the same problem category. Furthermore, while many works focus solely on accuracy, PHILDER also emphasizes computational efficiency, a crucial aspect for resource-constrained devices, which is not widely addressed in many of the compared studies. The combination of high accuracy and low per-sample latency (12.9811 ms) positions PHILDER as a robust and practical solution for production environments with resource limitations.

It can be observed in Table 3 that oversampling generally produced superior accuracy, precision, recall, and F1-score compared to undersampling and the original data; however, oversampling also incurred the highest computational cost for training, testing, and per-sample detection. Consequently, we proposed evaluating the models using the TES, IES, and RTDE metrics. These composite measures allow comparison of models by both predictive performance and computational efficiency, guiding the selection of the most appropriate architecture for real-time phishing detection on resource-constrained devices.

The oversampling approach was found to present three main issues: increased computational cost, risk of overfitting, and comparatively inferior robustness. By artificially expanding the training set—either by replicating minority-class instances or generating synthetic examples—oversampling significantly raises both training and inference times, undermining TES, IES, and RTDE, which are critical in constrained environments. Moreover, oversampling may encourage memorization over generalization, thereby masking true accuracy and weakening real-world phishing detection robustness.

While we acknowledge that parameter tuning for SMOTE could potentially mitigate some of these issues, our focus was on finding a solution that inherently balanced performance with the strict computational constraints of

TABLE 4. Statistical analysis of performance metrics (mean ± std).

Model	Accuracy	F1-score	TES	RTDE
TinyBERT (undersampling)	0.9927 ± 0.0006	0.9927 ± 0.0004	0.92 ± 0.01	0.93 ± 0.01
MiniLM (undersampling)	0.9889 ± 0.0007	0.9919 ± 0.0005	0.91 ± 0.01	0.91 ± 0.02
DistilBERT (undersampling)	0.9934 ± 0.0005	0.9932 ± 0.0006	0.89 ± 0.02	0.87 ± 0.03
ALBERT (undersampling)	0.9834 ± 0.0008	0.9831 ± 0.0007	0.82 ± 0.02	0.80 ± 0.02
MobileBERT (undersampling)	0.9904 ± 0.0004	0.9823 ± 0.0003	0.86 ± 0.02	0.85 ± 0.01

resource-limited devices. The observed empirical results, where oversampled models did not consistently outperform their counterparts trained on original or undersampled data, further supported our decision to prioritize efficiency and robustness over the potential benefits of fine-tuning a computationally intensive oversampling method. Empirically, oversampled models did not outperform their counterparts trained on original or undersampled data. TinyBERT and MiniLM with undersampling consistently surpassed all models in composite metrics (TES, IES, RTDE), achieving shorter times and similar or better accuracy.

To reinforce the consistency and reliability of the reported results, a statistical analysis was conducted across three independent runs for each model using the undersampling approach. Table 4 summarizes the mean and standard deviation for key performance metrics (Accuracy, F1-score, TES, and RTDE), providing further insight into the robustness and reproducibility of PHILDER's performance under realistic deployment conditions.

Regarding undersampling, while it inherently involves discarding data, our comprehensive evaluation using TES, IES, and RTDE metrics demonstrates that the chosen undersampling approach, particularly with TinyBERT, achieved an optimal balance between predictive performance and computational efficiency, making it highly suitable for resource-constrained environments without significant loss of critical information to address the issue at hand.

To further evaluate the robustness and contribution of key components in the PHILDER framework, an ablation study was performed. Two critical design aspects were analyzed: (i) the use of pretrained weights in the TinyBERT model and (ii) the application of the undersampling technique to address class imbalance.

Three configurations were considered for comparison:

- **PHILDER (full):** TinyBERT with pretrained weights and undersampling.
- **w/o undersampling:** TinyBERT with pretrained weights, without undersampling.
- **w/o pretraining:** TinyBERT initialized with random weights, using undersampling.

Table 5 presents the results for each configuration, including accuracy, F1-score, and the composite efficiency metrics TES, IES, and RTDE.

The results indicate that both components—pretrained weights and undersampling—contribute significantly to the overall performance and efficiency of the model. The absence of pretrained weights led to a notable decline in all evaluated metrics. Similarly, excluding undersampling reduced the F1-score due to poorer recall. These findings validate the importance of each design decision and reinforce the architectural choices implemented in PHILDER for deployment on resource-constrained devices.

TABLE 5. Ablation study results.

Configuration	Accuracy	F1-score	TES	IES	RTDE
PHILDER (full)	99.27%	0.993	0.92	0.89	0.93
w/o undersampling	96.82%	0.958	0.91	0.88	0.93
w/o pretraining	93.45%	0.925	0.80	0.76	0.84

Based on the TES, IES, and RTDE results, we decided to exclude the oversampling approach from the final analyses, focusing exclusively on configurations that balance predictive performance and computational efficiency. The performance results are illustrated in Figures 3, 4, and 5.

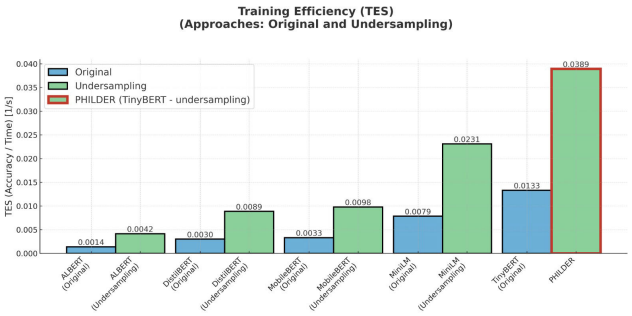


FIGURE 3. TES plot - Training efficiency.

Figure 3 shows that the PHILDER model—formerly identified as TinyBERT with the undersampling approach—is the most efficient in terms of training, achieving the highest ratio of accuracy to training time (TES) among all evaluated models. This makes it ideal for scenarios requiring frequent or rapid retraining, especially in environments with limited computational resources. The MiniLM model (undersampling) also performed excellently, offering an even lighter alternative, although with a TES slightly lower than PHILDER.

The MobileBERT, DistilBERT, and ALBERT models performed significantly worse than PHILDER and MiniLM in both the original and undersampling settings. Although their accuracy values are similar—ranging from 0.9804 to 0.9973 (Table 3)—their training times vary substantially, from 70.84 s to 702.01 s (Table 3), which explains the TES disparities. Hence, evaluating by TES is more informative,

since accuracy alone shows only marginal differences between the best and worst models.

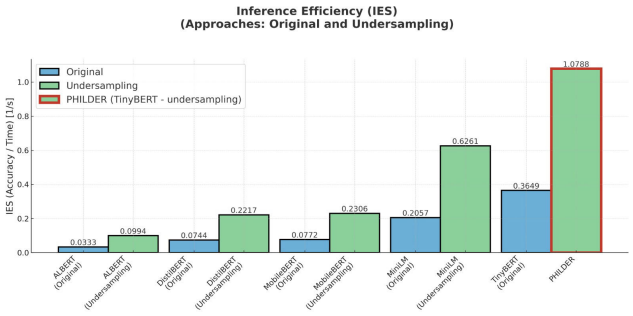


FIGURE 4. IES plot - Inference efficiency.

Figure 4 highlights that PHILDER is the most efficient during inference, achieving the highest ratio of accuracy to testing time (IES) among all models. This efficiency makes it especially suitable for systems performing frequent, large-scale checks, such as batch email filters or near-real-time detection pipelines. The MiniLM model (undersampling) also demonstrated strong performance, providing a lightweight and efficient alternative, albeit with an IES slightly below PHILDER.

Similarly to the TES results, ALBERT, DistilBERT, and MobileBERT underperformed compared to PHILDER and MiniLM, due to testing times ranging from 2.57 s to 29.60 s (Table 3). In this case, PHILDER not only achieved the highest accuracy but also the shortest testing time. Among the original-data models, TinyBERT likewise recorded the lowest testing time, which accounts for its significantly higher IES relative to other models.

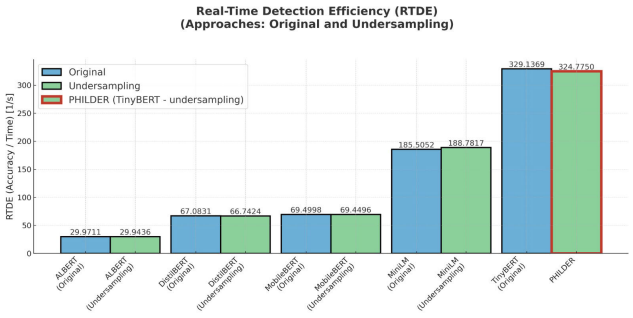


FIGURE 5. RTDE plot - Real-time detection efficiency.

Figure 5 shows that TinyBERT, using the original approach, achieved the highest real-time detection efficiency (RTDE), making it the preferred choice for applications that demand immediate responses, such as firewalls, email gateways, and embedded systems. PHILDER follows closely, with highly competitive performance and extremely low per-sample latency, underscoring its ability to operate effectively in resource-constrained, real-time detection environments. Models like MiniLM (original and undersampling) also exhibited good real-time efficiency, presenting viable options when extreme lightweight deployment is required. Conversely, the ALBERT family models showed the poorest RTDE performance, with high per-sample latencies that limit their use in real-time scenarios.

The decision to adopt TinyBERT was supported not only by its superior performance in predictive metrics but also by its practical advantages in constrained environments. TinyBERT is specifically designed to reduce the number of parameters and layers compared to other transformer-based architectures, which results in faster training, shorter inference times, and lower memory usage. These characteristics were reflected in the experimental results, where TinyBERT consistently achieved the best composite scores in TES, IES, and RTDE. Its ability to deliver high accuracy with minimal computational overhead makes it especially suitable for edge devices, mobile endpoints, and other limited-resource platforms.

As a result, the TinyBERT model with undersampling was adopted as the project's final solution—christened PHILDER—due to its excellent performance across all composite metrics and its low computational resource consumption, making it ideal for deployment in constrained production environments that closely resemble real-world applications.

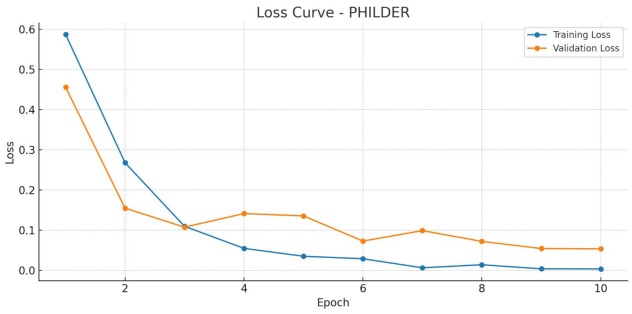


FIGURE 6. Loss curve for the PHILDER model.

Figure 6 presents the loss function of the PHILDER model. During epochs 1 and 2, the training loss remained above the validation loss curve, indicating that the model was still adapting. In epoch 3, both curves converged, marking the transition point between them. From epochs 4 through 10, the training loss fell below the validation loss, which is the expected behavior for a well-trained model and demonstrates that the model is properly fitted.

V. CONCLUSION

Phishing detection is an essential task in the current cybersecurity landscape, given the growing connectivity and the transmission of sensitive information across numerous networked devices. Developing improvements that enhance device security is critical, as cyberattacks become increasingly sophisticated and damaging. While attacks are growing more destructive and harder to detect, it is indispensable

to advance technologies that prevent, detect, and mitigate cybercrime. Thus, the different approaches tested in this study make a significant contribution to the literature by exploring deep learning-based alternatives and improvements for email phishing detection.

The primary contribution of this article is that efficient phishing detection requires more than comparing traditional metrics; it is essential to consider a model's suitability for real-world contexts, where high computational resources may not be available and real-time responses are often required. Therefore, a model must not only be accurate but also efficient to be truly applicable.

The results for the PHILDER model-based on the three composite metrics (TES, IES, and RTDE) — show that it is the most advantageous solution overall. PHILDER achieves the highest training efficiency (TES), the best batch inference performance (IES), and excellent real-time efficiency (RTDE), offering the optimal balance of accuracy, speed, and resource economy. This makes it highly recommended for phishing detection applications in hardware-constrained, real-time environments. MiniLM-family models also proved competitive, particularly in scenarios demanding extreme lightweight deployment and rapid inference.

As discussed, the ALBERT, DistilBERT, and MobileBERT models exhibited traditional metric values (accuracy, precision, recall, and F1-score) close to those of the best models (PHILDER and MiniLM), which could misleadingly suggest their suitability for deployment. However, achieving comparable performance with these models entails a high computational cost, rendering their use impractical in constrained environments.

For future research, it would be valuable to incorporate additional deep learning models for phishing detection and evaluate them using TES, IES, and RTDE, comparing their performance with the results of this study. Such efforts would significantly advance the cybersecurity field by identifying increasingly advanced and efficient solutions.

ACKNOWLEDGMENT

The authors gratefully acknowledge the Department of Electrical Engineering at the University of Brasília (UnB) for its institutional support and the computational infrastructure that was essential to the development of this work.

REFERENCES

[1] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Syst. Appl.*, vol. 117, pp. 345–357, Mar. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417418306067

[2] B. B. Gupta, N. A. G. Arachchilage, and K. E. Psannis, "Defending against phishing attacks: Taxonomy of methods, current issues and future directions," *Telecommun. Syst.*, vol. 67, no. 2, pp. 247–267, Feb. 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:31731180

[3] S. R. Curtis, P. Rajivan, D. N. Jones, and C. González, "Phishing attempts among the dark triad: Patterns of attack and vulnerability," *Comput. Hum. Behav.*, vol. 87, pp. 174–182, Jan. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0747563218302620

[4] O. A. Akanbi, I. S. Amiri, and E. Fazeldehkordi, Eds., "Chapter 5| implementation and result," in *A Machine-Learning Approach to Phishing Detection and Defense*. Boston, MA, USA: Syngress, 2015, pp. 55–73.

[5] IBM. (2024). *Cost of a Data Breach Report*. [Online]. Available: https://www.ibm.com/br-pt/reports/data-breach

[6] G. Vrbančič, I. Fister, and V. Podgorelec, "Swarm intelligence approaches for parameter setting of deep learning neural network: Case study on phishing websites classification," in *Proc. 8th Int. Conf. Web Intell., Mining Semantics*, Jun. 2018, pp. 1–8.

[7] A. Zamir, H. U. Khan, T. Iqbal, N. Yousaf, F. Aslam, A. Anjum, and M. Hamdani, "Phishing Web site detection using diverse machine learning algorithms," *Electron. Library*, vol. 38, no. 1, pp. 65–80, Mar. 2020.

[8] Y. A. Alsariera, A. V. Elijah, and A. O. Balogun, "Phishing website detection: Forest by penalizing attributes algorithm and its enhanced variations," *Arabian J. for Sci. Eng.*, vol. 45, no. 12, pp. 10459–10470, Dec. 2020.

[9] Anti-Phishing Working Group (APWG). (2025). *Relatório De Tendências. Acessado Em 21 De Janeiro De 2025*. [Online]. Available: https://apwg.org

[10] F. Munguia-Galeano, A.-H. Tan, and Z. Ji, "Deep reinforcement learning with explicit context representation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 419–432, Jan. 2025.

[11] E. S. Gualberto, R. T. De Sousa, T. P. De B. Vieira, J. P. C. L. Da Costa, and C. G. Duque, "From feature engineering and topics models to enhanced prediction rates in phishing detection," *IEEE Access*, vol. 8, pp. 76368–76385, 2020.

[12] G. D. Bispo, G. F. Vergara, G. M. Saiki, P. H. D. S. Martins, J. G. Coelho, G. A. P. Rodrigues, M. N. D. Oliveira, L. R. Mosquéra, V. P. Gonçalves, C. Neumann, and A. L. M. Serrano, "Automatic literature mapping selection: Classification of papers on industry productivity," *Appl. Sci.*, vol. 14, no. 9, p. 3679, Apr. 2024.

[13] K. L. Chiew, C. L. Tan, K. Wong, K. S. C. Yong, and W. K. Tiong, "A new hybrid ensemble feature selection framework for machine learning-based phishing detection system," *Inf. Sci.*, vol. 484, pp. 153–166, May 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025519300763

[14] Y. A. Alsariera, V. E. Adeyemo, A. O. Balogun, and A. K. Alazzawi, "AI meta-learners and extra-trees algorithm for the detection of phishing websites," *IEEE Access*, vol. 8, pp. 142532–142542, 2020.

[15] A. Almomani, M. Alauthman, M. T. Shatnawi, M. Alweshah, A. Alrosan, W. Alomoush, B. B. Gupta, B. B. Gupta, and B. B. Gupta, "Phishing website detection with semantic features based on machine learning classifiers: A comparative study," *Int. J. Semantic Web Inf. Syst.*, vol. 18, no. 1, pp. 1–24, Feb. 2022.

[16] A. A. Ubing, S. Kamilia, A. Abdullah, N. Jhanjhi, and M. Supramaniam, "Phishing website detection: An improved accuracy through feature selection and ensemble learning," *Int. J. Adv. Comput. Sci. Appl.*, vol. 10, no. 1, pp. 1–6, 2019.

[17] G. Mujtaba, L. Shuib, R. G. Raj, N. Majeed, and M. A. Al-Garadi, "Email classification research trends: Review and open issues," *IEEE Access*, vol. 5, pp. 9044–9064, 2017.

[18] G. De La Torre Parra, P. Rad, K.-K.-R. Choo, and N. Beebe, "Detecting Internet of Things attacks using distributed deep learning," *J. Netw. Comput. Appl.*, vol. 163, Aug. 2020, Art. no. 102662. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804520301363

[19] A. Alhogail and A. Alsabih, "Applying machine learning and natural language processing to detect phishing email," *Comput. Secur.*, vol. 110, Nov. 2021, Art. no. 102414. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404821002388

[20] L. Tang and Q. H. Mahmoud, "A deep learning-based framework for phishing website detection," *IEEE Access*, vol. 10, pp. 1509–1521, 2022.

[21] L. Halgaš, I. Agrafiotis, and J. R. C. Nurse, "Catching the phish: Detecting phishing attacks using recurrent neural networks (RNNs)," in *Proc. 20th Int. Conf. Inf. Secur. Appl.*, Jeju Island, South Korea, Aug. 2020, pp. 219–233.

[22] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2018, pp. 4171–4186.

[23] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[24] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.

[25] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling BERT for natural language understanding," 2019, *arXiv:1909.10351*.
[26] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: A compact task-agnostic BERT for resource-limited devices," 2020, *arXiv:2004.02984*.
[27] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
[28] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5776–5788.



GUILHERME DANTAS BISPO received the bachelor's degree in computer science from the Catholic University of Brasília (UCB), the Post-graduate degree in software production with an emphasis on free software from the Federal University of Lavras (UFLA), and the Professional master's degree in data science from the Applied Computing Graduate Program (PPCA), University of Brasília (UnB). He is currently pursuing the Ph.D. degree in electrical engineering with UnB, with a research focus on distributed systems and communication networks. He is a Federal Civil Servant at the Superior Labor Court (TST), working as a Software Architect. He is also a Researcher with UnB and a member with the PROJECTUM Research Group. His expertise includes software architecture, system development, DevOps practices, and public sector innovation.



CÉSAR AUGUSTO BORGES DE ANDRADE received the bachelor's degree in data processing from the Mackenzie Presbyterian University, São Paulo, Brazil, in 1997, and the M.Sc. degree in systems and computing from the Military Institute of Engineering (IME), Rio de Janeiro, Brazil, in 2013. He is currently pursuing the Ph.D. degree with the Graduate Program in Electrical Engineering, University of Brasília (UnB), researching machine learning applied to malicious software detection.



GABRIELA MAYUMI SAIKI received the bachelor's degree in production engineering from the University of Brasília (UnB), in 2023, where she was recognized as the top student in her graduating class, where she is currently pursuing the master's degree with the Professional Graduate Program in Electrical Engineering (PPEE). She is a member with the PROJECTUM Research Group, where she conducts research focused on advanced efficiency techniques, particularly data envelopment analysis (DEA), and data analysis and data security. She has participated in projects evaluating the efficiency of strategic sectors of Brazilian economy, contributing to performance measurement methodologies and management enhancement practices.



RAQUEL VALADARES BORGES is currently pursuing the bachelor's degree in statistics with the University of Brasília (UnB). She is also a contributor to the PROJECTUM–UnB Research Group and a volunteer at the Institutional Program for Scientific Initiation Scholarships (PIBIC–UnB), with research focused on birth rate characteristics in the Metropolitan Area of Brasília and maternal mortality risks. She also works as a Junior Researcher with the Fundação Getúlio Vargas' Public Policy Experimental Laboratory (FGV–LEPP) and a Business Intelligence Analyst with Núcleo de Apoio CAIXA.



ANDRÉ LUIZ MARQUES SERRANO received the bachelor's degree in mathematics from the University of Brasília (UnB), in 1999, the Post-graduate degree in data science from the Pontifical Catholic University of Rio de Janeiro (PUC-Rio), and the M.Sc. and Ph.D. degrees in economics with an emphasis on statistical modeling from UnB), in 2008 and 2012, respectively. He is currently an Associate Professor with the Department of Production Engineering, UnB, and a Faculty Member with the Professional Graduate Program in Electrical Engineering (PPEE/UnB). He coordinates the PROJECTUM Laboratory (LinceX) for Computational Intelligence and User Experience and is the Deputy Head of the Department of Production Engineering (2025–2027). His research interests include data analysis, machine learning, multivariate analysis, DEA, and time series modeling.



GERALDO PEREIRA ROCHA FILHO received the M.Sc. and Ph.D. degrees in computer science and computational mathematics from the University of São Paulo (ICMC/USP). He completed postdoctoral research at UNICAMP. He coordinates the LaRSI Laboratory and has led projects funded by CNPq, CAPES, and FAPESP. He is currently a Professor with the Department of Exact and Technological Sciences, State University of Southwest Bahia (UESB), and a Former Faculty Member with the University of Brasília (UnB). His research interests include networks, intelligent systems, and distributed computing. He served as the General Chair for SBRC 2023 and received several academic awards.



VINÍCIUS PEREIRA GONÇALVES received the B.Sc. degree in computer science and mathematics education from Parà institutions, and the M.Sc. degree in computer science and the Ph.D. degree in computer science and computational mathematics from the University of São Paulo (ICMC/USP), in 2012 and 2016, respectively, with a doctoral internship at the University of Arizona, from 2014 to 2015. He completed a Postdoctoral Fellowship at the School of Medicine, USP, in collaboration with UNIFACCAMP, and was a Visiting Researcher with the University of Coimbra, in 2019. He is currently an Assistant Professor with the Department of Electrical Engineering, UnB, coordinator of the communication networks engineering undergraduate program, and a member of graduate programs in electrical engineering. He is a Researcher at the AQUARELA Group and serves as the Regional Secretary for Brazilian Computer Society (SBC). His research interests include HCI, the IoT, cybersecurity, machine learning, and mobile health.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614